

## **Arquitectura de Firmware por Capas**

La arquitectura de firmware por capas es una metodología de diseño que organiza el software embebido en niveles jerárquicos, donde cada capa tiene una responsabilidad específica y utiliza únicamente los servicios de la capa inferior. Su objetivo es mejorar la modularidad, reutilización, mantenimiento y escalabilidad del sistema.

### **Capas principales**

#### **1. Hardware**

Corresponde a los elementos físicos del sistema:

- Microcontrolador (ESP32, STM32, etc.).
- GPIO.
- ADC.
- UART.
- SPI.
- I2C.
- Sensores y actuadores.

#### **2. Drivers o HAL (Hardware Abstraction Layer)**

Encapsulan el acceso al hardware mediante funciones sencillas, ocultando detalles de registros y periféricos.

Ejemplos:

- `led_on()`
- `led_off()`
- `adc_read()`
- `uart_send()`

#### **3. Servicios o Middleware**

Implementan funcionalidades más complejas utilizando uno o varios drivers.

Ejemplos:

- Lectura y filtrado de temperatura.

- Comunicación Bluetooth Low Energy (BLE).
- Protocolos de comunicación.
- Manejo de almacenamiento.

#### **4. Aplicación**

Contiene la lógica principal del sistema y utiliza los servicios disponibles para cumplir los objetivos del proyecto.

Ejemplo:

- Encender un LED cuando la temperatura supera un umbral.
- Enviar datos por BLE.
- Controlar un actuador según la lectura de sensores.

#### **Flujo de funcionamiento**

Aplicación → Servicios → Drivers/HAL → Hardware

Ejemplo:

Aplicación

↓

Servicio de temperatura

↓

Driver ADC

↓

ADC del microcontrolador

↓

Sensor de temperatura

#### **Ventajas**

- Mayor organización del código.
- Facilita el mantenimiento y la depuración.
- Permite reutilizar módulos en diferentes proyectos.
- Reduce el acoplamiento entre hardware y aplicación.
- Facilita la escalabilidad del sistema.

#### **Estructura típica de carpetas**

project/

|

| └─ main/

|   └─ main.c

|

| └─ drivers/

|   └─ led.c

|   └─ led.h

|   └─ adc.c

|   └─ adc.h

|

| └─ services/

|   └─ ble.c

|   └─ ble.h

|   └─ temperature.c

|   └─ temperature.h

|

| └─ include/

## Conclusión

La arquitectura de firmware por capas separa claramente las responsabilidades del sistema embebido, permitiendo desarrollar aplicaciones más robustas, mantenibles y reutilizables. Es una de las arquitecturas más utilizadas en proyectos profesionales de sistemas embebidos.